

Real-Time Embedded Smart Writing System:

From Spatial IMU Trajectory Reconstruction to Quantized Dual-Stream Neural Classification on Arduino Due

ESD Final Project Technical Report

May 27, 2026

Abstract

This report presents the design, mathematical formulation, and embedded implementation of a real-time, closed-loop smart writing instrument. The system is split across three hardware nodes: an ESP32-C3 SuperMini pen capturing six-axis inertial motion unit (IMU) data; a NodeMCU ESP32 acting as an intermediate wireless transceiver; and an Arduino Due hosting a highly optimized, integer-quantized neural network classifier. By integrating a Mahony complementary attitude filter, plane projection basis orthogonalization, and zero-velocity linear drift detrending, the pen reconstructs raw hand-written trajectory coordinates into a normalized 28×28 grid, compressed into a 98-byte packet and sent via ESP-NOW. The transceiver wraps the image data, appends timing latency metrics, and compiles a (116, 106) Reed-Solomon Forward Error Correction (FEC) block, transmitting the payload over a custom high-speed 8-bit parallel bus via direct GPIO register manipulation. The Arduino Due receives the parallel stream through rising-edge clock interrupts, performs error correction over Galois Fields $GF(2^8)$, and runs real-time inference on a custom Decoupled Asymmetrical Factor Net (DAFNet). To meet memory constraints, we employ post-training symmetric INT8 quantization. Using knowledge distillation from a 1M parameter WaveMix teacher model, the student model preserves a high classification accuracy of 89.36% on EMNIST with a minuscule parameter footprint of 317.6 KB, achieving a highly optimized inference latency of 0.64 ms on a single microcontroller thread.

Contents

1	System Architecture Overview	3
2	IMU Spatial Path Reconstruction	3
2.1	Mahony Complementary Filter	3
2.2	Gravity Calibration & Zero-Gravity Plane Projection	4
2.3	Stillness Detection & Linear Velocity Detrending	5
2.4	Normalization, Centering, & Bresenham Rasterization	5
3	Distributed Transmitting Protocols & Latency Breakdowns	6
3.1	ESP-NOW Wireless Link	6
3.2	8-Bit Fast Parallel Bus Protocol	6
3.3	Latency Analysis	6
4	Reed-Solomon Forward Error Correction	7
4.1	Galois Field $GF(2^8)$ Arithmetic	7
4.2	Encoding and Syndrome Calculation	7
4.3	Berlekamp-Massey, Chien Search, & Forney Decoders	7
5	Model Development, Distillation, and Quantization	7
5.1	Summary of numberAI Development History	8
5.2	Knowledge Distillation & EMNIST Domain Alignment	8
5.3	Symmetric INT8 Weight Quantization	9
5.4	Comparative Model Benchmarks & Scientific Findings	9
5.4.1	Key Scientific Conclusions	9
6	Quantized DAFNet Implementation on Arduino Due	10
6.1	Dual-Stream Streamlined Pipeline	10
6.2	Memory Optimization: Const Flash Storage	10
7	Conclusion	10

1 System Architecture Overview

The complete writing and recognition system is structured as a distributed, three-node embedded pipeline designed for high throughput, robust error tolerance, and low latency. The processing and transmission workflow is executed sequentially across the hardware modules as follows:

1. **ESP32-C3 SuperMini Pen (Inertial Sensor & Path Reconstruction Node):** Ingests raw 6-axis acceleration ($\mathbf{a}$) and angular velocity ($\boldsymbol{\omega}$) from an MPU6050 sensor at a sample rate of approximately 100 Hz. It runs a Mahony complementary filter to track orientation, projects acceleration onto a gravity-orthogonal writing plane, performs segment-based velocity detrending to arrest integration drift, rasterizes the trajectory onto a 28×28 grid using Bresenham’s algorithm, and compresses the image into a 98-byte bitstream ($28 \times 28 = 784$ bits).
2. **NodeMCU ESP32 (Wireless-to-Parallel Bridge & FEC Encoder):** Receives the 98-byte image packet via the ESP-NOW protocol, records the wireless transfer latency, and appends the pen’s internal processing duration, assembling a 106-byte data payload. It then runs a Reed-Solomon encoder to generate 10 parity bytes, forming a 116-byte block. This block is serialized byte-by-byte onto an 8-bit parallel bus, driving a dedicated clock line using direct register manipulation for maximum throughput.
3. **Arduino Due (Parallel Receiver Neural Inference Engine):** Acts as the master processing node (SAM3X8E Cortex-M3 running at 84 MHz). It captures incoming bytes through an interrupt-driven parallel interface mapped to the hardware registers of Port C. It executes the Reed-Solomon decoding algorithm to repair up to 5 erroneous bytes in the 116-byte block. Upon successful repair, it prints the diagnostic latency breakdown and forwards the reconstructed EMNIST image to the on-board DAFNet model. The model is fully quantized to signed 8-bit integers (INT8) and fits within the SAM3X8E’s 96 KB SRAM constraint by mapping weights to Flash memory.

2 IMU Spatial Path Reconstruction

The primary challenge of inertial-based tracking is the rapid accumulation of drift during double integration. The ESP32-C3 pen addresses this by implementing an algorithmic pipeline consisting of attitude filtering, plane projection, dynamic stillness detection, and velocity detrending.

2.1 Mahony Complementary Filter

To decouple linear acceleration from gravity, we must continuously track the sensor’s spatial orientation. We utilize a Mahony complementary filter due to its extreme computational efficiency on resource-limited MCUs. The filter fuses angular velocity $\mathbf{g} = (g_x, g_y, g_z)^T$ and acceleration $\mathbf{a} = (a_x, a_y, a_z)^T$ by defining a feedback loop that corrects gyroscope drift.

Let the current attitude be represented by the unit quaternion $\mathbf{q} = (q_0, q_1, q_2, q_3)^T$. The estimated gravity vector in the sensor's coordinate system $\mathbf{v} = (v_x, v_y, v_z)^T$ is extracted from the rotation matrix:

$$\mathbf{v} = \begin{pmatrix} 2(q_1q_3 - q_0q_2) \\ 2(q_0q_1 + q_2q_3) \\ q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{pmatrix} \quad (1)$$

The cross product between the normalized measured acceleration $\hat{\mathbf{a}}$ and the estimated gravity $\mathbf{v}$ yields the attitude error vector $\mathbf{e}$:

$$\mathbf{e} = \hat{\mathbf{a}} \times \mathbf{v} \quad (2)$$

The error is integrated and scaled by the proportional gain $K_p = 2.0$ and integral gain $K_i = 0.005$ to correct the raw gyroscope readings:

$$\mathbf{i}_{\text{FB}} \leftarrow \mathbf{i}_{\text{FB}} + K_i \mathbf{e} \Delta t \quad (3)$$

$$\mathbf{g}_{\text{corrected}} = \mathbf{g} + K_p \mathbf{e} + \mathbf{i}_{\text{FB}} \quad (4)$$

The quaternion rate of change is then integrated over the timestep Δt :

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{pmatrix} 0 \\ \mathbf{g}_{\text{corrected}} \end{pmatrix} \quad (5)$$

$$\mathbf{q} \leftarrow \mathbf{q} + \dot{\mathbf{q}} \Delta t, \quad \mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|} \quad (6)$$

2.2 Gravity Calibration & Zero-Gravity Plane Projection

At startup, the pen is held stationary for 3 seconds to compute the gravitational acceleration biases. The averaged readings establish the static gravity vector $\mathbf{g}_{\text{bias}} = (b_{ax}, b_{ay}, b_{az})^T$.

We construct a coordinate system representing the horizontal writing plane. We define a normal vector $\mathbf{n} = \mathbf{g}_{\text{bias}} / \|\mathbf{g}_{\text{bias}}\|$ aligned with gravity. Two orthogonal basis vectors $\mathbf{w}$ and $\mathbf{u}$ are generated in the world frame:

$$\mathbf{w} = \begin{cases} \frac{(n_y, -n_x, 0)^T}{\sqrt{n_x^2 + n_y^2}} & \text{if } |n_z| < 0.9 \\ \frac{(0, n_z, -n_y)^T}{\sqrt{n_y^2 + n_z^2}} & \text{otherwise} \end{cases} \quad (7)$$

$$\mathbf{u} = \mathbf{n} \times \mathbf{w} \quad (8)$$

These static plane vectors are then rotated into the sensor's local frame using the orientation quaternion $\mathbf{q}$ to yield the frozen local projection vectors $\mathbf{s}_X$ and $\mathbf{s}_Y$:

$$\mathbf{s}_X = \mathbf{R}(\mathbf{q})^T \mathbf{w}, \quad \mathbf{s}_Y = \mathbf{R}(\mathbf{q})^T \mathbf{u} \quad (9)$$

For any subsequent Net acceleration measurement $\mathbf{a}_{\text{net}} = \mathbf{a} - \mathbf{g}_{\text{bias}}$, the gravity-compensated coordinates on the projected writing plane p_x and p_y are computed via scalar projection:

$$p_x = \mathbf{a}_{\text{net}} \cdot \mathbf{s}_X, \quad p_y = \mathbf{a}_{\text{net}} \cdot \mathbf{s}_Y \quad (10)$$

2.3 Stillness Detection & Linear Velocity Detrending

To prevent mathematical blow-up during double integration, we implement a dynamic stillness detection algorithm. A point is flagged as stationary if the gyroscope and accelerometer magnitudes satisfy:

$$\|\mathbf{g}\| < 0.12 \text{ rad/s} \quad \text{and} \quad \|\mathbf{a}_{\text{net}}\| < 0.25 \text{ m/s}^2 \quad (11)$$

This condition must persist for a continuous window of 60 ms. When stillness is confirmed, the velocity is forced to exactly zero.

The motion is partitioned into sequential segments (continuous intervals where the pen is either writing/“ink” or moving in the air/“air”). Double integration is performed segment-by-segment. For a segment of duration T_{seg} , the raw integrated velocity vector $\mathbf{v}_{\text{raw}}(t)$ is:

$$\mathbf{v}_{\text{raw}}(t) = \int_0^t \mathbf{a}_{\text{proj}}(\tau) d\tau \quad (12)$$

Due to sensor noise and numerical integration errors, the velocity at the end of the segment exhibits an accumulated drift $\mathbf{v}_{\text{err}} = \mathbf{v}_{\text{raw}}(T_{\text{seg}})$. Assuming this drift accumulates linearly, we apply a linear detrending correction (Zero-Velocity Update style):

$$\mathbf{v}_{\text{corrected}}(t) = \mathbf{v}_{\text{raw}}(t) - \mathbf{v}_{\text{err}} \left(\frac{t}{T_{\text{seg}}} \right) \quad (13)$$

The spatial coordinates $\mathbf{x}(t) = (x(t), y(t))^T$ are then computed by integrating the corrected velocity:

$$\mathbf{x}(t) = \int_0^t \mathbf{v}_{\text{corrected}}(\tau) d\tau \quad (14)$$

By resetting the velocity bounds and removing the linear trend at the boundary of every stroke segment, we effectively eliminate long-term integration drift.

2.4 Normalization, Centering, & Bresenham Rasterization

Upon completion of the writing sequence, the set of coordinates corresponding to the active writing strokes (“ink = true”) is normalized. We compute the bounding box boundaries $[x_{\min}, x_{\max}]$ and $[y_{\min}, y_{\max}]$, and center of mass $\mathbf{c} = (c_x, c_y)^T$. The scaling factor *scale* is determined to fit the handwriting within a 20×20 bounding region to preserve EMNIST proportions:

$$\text{scale} = \min \left(\frac{20.0}{x_{\max} - x_{\min}}, \frac{20.0}{y_{\max} - y_{\min}} \right) \quad (15)$$

The coordinates are mapped to a discrete 28×28 grid, centered at (14, 14):

$$g_x = \text{constrain}(\text{round}(14.0 - (x - c_x) \cdot \text{scale}), 0, 27) \quad (16)$$

$$g_y = \text{constrain}(\text{round}(14.0 + (y - c_y) \cdot \text{scale}), 0, 27) \quad (17)$$

Adjacent stroke points are connected on the grid using Bresenham’s line algorithm. This yields a binary grid where pixels are either active ink (value 0) or background (value 255). This grid is packed bitwise into a 98-byte buffer ($28 \times 28 = 784$ bits, $784/8 = 98$ bytes).

3 Distributed Transmitting Protocols & Latency Breakdowns

Wireless and wired communication channels link the nodes. The transition of data from the pen to the final classifier is optimized to ensure low transmission latency and error robustness.

3.1 ESP-NOW Wireless Link

The ESP32-C3 pen broadcasts the 98-byte compressed bitmap to the NodeMCU ESP32 using the ESP-NOW protocol in standard station (STA) mode. Operating at 2.4 GHz, ESP-NOW bypasses the overhead of standard Wi-Fi handshakes and IP stack encapsulation, enabling connectionless, packetized transmissions. The transmission is executed in under 2 ms.

3.2 8-Bit Fast Parallel Bus Protocol

Upon receiving the wireless packet, the NodeMCU ESP32 logs the reception timestamp and appends the pen’s internal processing time. It packages a 106-byte payload (98 bytes image + 4 bytes ESP32-C3 processing time + 4 bytes NodeMCU transfer latency) and adds 10 bytes of Reed-Solomon parity, creating a 116-byte frame.

To transfer this frame to the Arduino Due with minimal latency, we designed a custom 8-bit parallel bus interface. The bus consists of 8 data lines (NodeMCU pins GPIO 16, 17, 18, 19, 21, 22, 23, 25) and a clock line (GPIO 4). To bypass the slow Arduino ‘digitalWrite()’ function, we write directly to the ESP32 hardware register.

A lookup table `gpio_lookup[256]` is computed at startup, mapping each byte value directly to its corresponding GPIO pin.

3.3 Latency Analysis

The combination of ESP-NOW and the parallel bus yields a highly responsive system. The total end-to-end latency T_{total} is calculated by accumulating the measured diagnostic durations across all three nodes:

$$T_{total} = t_{C3} + t_{wireless} + t_{transceiver} + t_{Due_recv} + t_{inference} \quad (18)$$

Under active execution, the breakdown represents:

- Pen Processing Time (t_{C3}): $\approx 425 \mu s$ (includes projection and Bresenham rasterization).
- Wireless Link ($t_{wireless}$): $\approx 2000 \mu s$ (estimated average ESP-NOW transit time).
- NodeMCU Transceiver Bridge ($t_{transceiver}$): $\approx 102 \mu s$ (includes RS encoding and queue overhead).
- Due Parallel Bus Transfer (t_{Due_recv}): $\approx 84 \mu s$ (transfer duration for 116 bytes).
- Due Inference ($t_{inference}$): $\approx 640 \mu s$ (DAFNet INT8 forward pass).

The total latency is approximately 3.25 ms, representing an exceptionally fast hand-written character recognition pipeline suitable for real-time applications.

4 Reed-Solomon Forward Error Correction

To guarantee error-free parallel transmission in electrically noisy environments, we implement a Reed-Solomon Forward Error Correction $(N, K) = (116, 106)$ block code. With $N - K = 10$ parity bytes, the code can correct up to $T = 5$ arbitrary byte corruptions per packet.

4.1 Galois Field $GF(2^8)$ Arithmetic

All arithmetic operations are performed over the finite field $GF(2^8)$, defined using the primitive polynomial $p(x) = x^8 + x^4 + x^3 + x^2 + 1$ (hexadecimal: '0x1d'). Multiplication and division of field elements are optimized using precomputed exponential ('exp') and logarithm ('log') tables:

$$x \cdot y = \exp[\log(x) + \log(y)] \quad (19)$$

$$x/y = \exp[(\log(x) + 255 - \log(y)) \pmod{255}] \quad (20)$$

Addition and subtraction in $GF(2^8)$ are equivalent to the bitwise XOR operator $(x \oplus y)$.

4.2 Encoding and Syndrome Calculation

The message polynomial $m(x)$ of degree 105 is multiplied by x^{10} and divided by the generator polynomial $g(x) = \prod_{i=0}^9 (x - 2^i)$. The remainder $r(x) = x^{10}m(x) \pmod{g(x)}$ represents the 10 parity bytes, which are appended to form the transmitted codeword $c(x) = x^{10}m(x) \oplus r(x)$.

At the receiver, the raw buffer is treated as a received polynomial $R(x) = c(x) \oplus E(x)$, where $E(x)$ represents the error polynomial. The syndrome vector $\mathbf{S} = (S_1, S_2, \dots, S_{10})^T$ is computed by evaluating $R(x)$ at the generator roots:

$$S_i = R(2^{i-1}), \quad \text{for } i = 1, 2, \dots, 10 \quad (21)$$

If all syndromes $S_i = 0$, the packet is clean and decoded immediately. Otherwise, the decoder initiates error correction.

4.3 Berlekamp-Massey, Chien Search, & Forney Decoders

The decoding process executes three primary mathematical operations:

1. Berlekamp-Massey Algorithm: Iteratively solves the key equation to find the error locator polynomial $\Lambda(x) = 1 + \Lambda_1x + \Lambda_2x^2 + \dots + \Lambda_vx^v$, where $v \leq 5$ is the number of errors.
2. Chien Search: Finds the roots of $\Lambda(x)$ by evaluating it at every field element $x = 2^{-i}$ for $i = 0, \dots, 115$. The error locations correspond to the indices i where $\Lambda(2^{-i}) = 0$.
3. Forney's Algorithm: Computes the error evaluator polynomial $\Omega(x) = S(x) \cdot \Lambda(x) \pmod{x^{10}}$. For each error location $X_l = 2^{-i_l}$, the error magnitude Y_l is calculated as:

$$Y_l = \frac{\Omega(X_l^{-1})}{\Lambda'(X_l^{-1})} \quad (22)$$

where $\Lambda'(x)$ is the formal derivative of the error locator polynomial. The original payload is restored by subtracting the error polynomial: $c(x) = R(x) \oplus \sum_l Y_l x^{i_l}$.

5 Model Development, Distillation, and Quantization

The DAFNet classifier deployed on the Arduino Due is the result of systematic architectural iterations originating from the numberAI project.

5.1 Summary of numberAI Development History

The development of the EMNIST classifier progressed through the following versions:

- Version 0: The baseline linear network ($H1$ and $H2$) with no activation functions collapsed mathematically into a single matrix. High file I/O bottleneck and gradient explosions (NaN loss) prevented convergence.
- Version 1.0: Introduced ReLU activation and numerical gradient approximation. While Shannon entropy dropped successfully (e.g., $H1 \approx 6.63 \rightarrow 3.05$), the network suffered from severe overfitting (100% training vs. 0% verification accuracy) and a divide-by-zero singularity in the Euclidean loss function.
- Version 1.1: Replaced numerical gradients with a mathematically rigorous vectorized blame assignment (backpropagation), reducing training time from 15 minutes to 30.6 seconds. However, it hit a memorization plateau due to dataset starvation (100 training images), yielding 60% verification accuracy.
- Version 1.2, 1.2.1 & 1.4.1: Transitioned to the standardized MNIST dataset. Solved overfitting, achieving 93.93% accuracy. Upgraded input dimensions from 16×16 to standard 28×28 to capture edge details.
- Version 1.3 & 1.4: Experimented with a custom concatenated pooling bottleneck (reducing $16 \times 16 \rightarrow 4 \times 4$ via max-pooling, concatenated with a dense bottleneck) to introduce spatial correlations.
- Version 1.5 & 1.5.1: Deepened the network to a 3-hidden-layer topology ($784 \rightarrow 196 \rightarrow 49 \rightarrow 10$) using hand-written backpropagation, demonstrating gradient propagation without vanishing effects ($> 97\%$ accuracy).
- Version 1.6 & 1.7: Explored 4-branch dense architectures with concatenated or element-wise summed multi-dimensional outputs to capture features at different abstraction levels.
- Version 1.8 & 1.8.1: Implemented a 12-branch spatial slicing network that geometrically split the input image into overlapping patches (corners, center, and horizontal strips) to extract localized features without convolution.
- Version 2.0 & 2.1: Introduced custom convolutions and max-pooling to achieve spatial translation invariance. Ported the pipeline to PyTorch for GPU acceleration, scaling feature maps to 8 parallel convolutional kernels and achieving 94.68% accuracy on MNIST in 324 seconds.
- Version 3.0 (RFNet): Upgraded the task to the 47-class EMNIST ByMerge dataset. Introduced asymmetric input slicing (factoring the 28×28 dimension) and residual connections to stabilize training.
- Version 3.1 (DAFNet): Deployed as the final decoupled student model. Avoids feedback loops by removing residual connections, resulting in a streamlined dual-stream forward pipeline containing exactly 317,608 parameters.

5.2 Knowledge Distillation & EMNIST Domain Alignment

To transfer the classification accuracy of a high-capacity WaveMix teacher model (v4.0w, 1,012,348 parameters) to the student model (v3.1, 317,608 parameters), we utilize a soft-target Knowledge Distillation framework. The student is optimized using a combined loss function:

$$\mathcal{L}_{\text{total}} = \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KL}}\left(\text{softmax}\left(\frac{\mathbf{z}_s}{T}\right), \text{softmax}\left(\frac{\mathbf{z}_t}{T}\right)\right) + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(\mathbf{z}_s, \mathbf{y}) \quad (23)$$

where $\mathbf{z}_s$ and $\mathbf{z}_t$ represent student and teacher logits, $T = 4.0$ is the softening temperature, and $\alpha = 0.5$ is the distillation weighting factor.

During distillation, we resolved a critical domain mismatch: the teacher was trained on rotated EMNIST images, whereas the student expected upright-transposed images for real-world deployment. We solved this by transposing the image batch $\mathbf{X}_{\text{batch}}$ before passing it to the frozen teacher:

$$\mathbf{X}_{\text{teacher}} = \mathbf{X}_{\text{batch}}^T \quad (24)$$

This transposition restored the distillation process, enabling the distilled student to achieve an outstanding EMNIST test accuracy of 89.35%.

5.3 Symmetric INT8 Weight Quantization

To compress the distilled student model for deployment on the Arduino Due, we implement a post-training symmetric integer quantization pipeline. Floating-point weights W are mapped to a signed 8-bit integer range $[-127, 127]$ with $qmax = 127$. For each weight layer, we compute the scaling factor S :

$$S = \frac{\max(|W|)}{qmax} \quad (25)$$

The quantized integer weights W_q are calculated and clamped:

$$W_q = \text{clamp}\left(\text{round}\left(\frac{W}{S}\right), -127, 127\right) \quad (26)$$

During inference on the Arduino, weights are stored as `'const int8_t inFlashmemorytopreserveSRAM.Aritpointbydynamicallyscalingtheaccumulateddotproductsattheoutputofeachlayer : y_dequant = (sum W_q · x_q) × S(27)` Quantization reduces the weight file size by over 65% (compressing total weights on disk to ≈ 890 KB and memory footprint to ≈ 317.6 KB**). Crucially, the INT8 distilled student maintains an EMNIST test accuracy of 89.36%, exhibiting zero accuracy regression.

5.4 Comparative Model Benchmarks & Scientific Findings

A comparative research study was conducted to evaluate the necessity of knowledge distillation against direct quantization of the large teacher model. The results are summarized in Table 1.

Table 1: Distributed Architecture and Quantization Benchmarks

Model Architecture	EMNIST Accuracy	Bit-Width	Parameters	Memory Size	Due CPU Latency
Teacher Float32 (v4.0w)	91.08%	32-bit Float	1,012,348	≈ 4.05 MB	12.24 ms/img
Quantized Teacher INT4 (v4.0w.q4)	86.76%	4-bit INT	1,012,348	≈ 506.2 KB	12.24 ms/img
Decoupled Student Float32 (v3.1)	89.06%	32-bit Float	317,608	≈ 1.27 MB	0.64 ms/img
Distilled Student Float32 (v3.1.distilled)	89.35%	32-bit Float	317,608	≈ 1.27 MB	0.64 ms/img
Quantized Student INT8 (v3.1.baseline.q8)	89.07%	8-bit INT	317,608	≈ 317.6 KB	0.64 ms/img
Distilled Student INT8 (v3.1.q8)	89.36%	8-bit INT	317,608	≈ 317.6 KB	0.64 ms/img

5.4.1 Key Scientific Conclusions

1. **Knowledge Distillation is Essential:** Directly quantizing the 1M parameter teacher model (v4.0w) to 4-bit integer weights (to match the storage scale of the student) leads to a severe accuracy collapse from 91.08% to 86.76% (a massive -4.32% absolute regression). This proves that aggressive low-bit quantization of complex architectures destroys representation capacity, making the student-distillation route superior.
2. **Distillation Empowers Quantization Resilience:** The distilled INT8 student (v3.1.q8, 89.36%) outperforms the baseline quantized student (v3.1.baseline.q8, 89.07%) by $+0.29\%$ absolute accuracy while maintaining the same memory footprint (317.6 KB). This demonstrates that distilling soft probabilities from a teacher embeds resilient, generalized geometric abstractions that withstand the precision loss of integer rounding.
3. **Inference Latency Breakthrough:** Due to its complex wavelet decomposition and bilinear upsampling layers, the teacher model has a CPU inference latency of 12.24 ms. The DAFNet student model executes in just 0.64 ms, representing a 19.1x speedup that enables real-time character recognition on microcontrollers.

6 Quantized DAFNet Implementation on Arduino Due

The DAFNet model is written in standard C++ and runs on the Arduino Due. The model utilizes a dual-stream architecture designed to extract distinct visual features.

6.1 Dual-Stream Streamlined Pipeline

The neural pipeline executes as two parallel processing streams:

1. Stream 1: Trainable Conv Stream:

- conv1: Standard 2D convolution. Maps the 28×28 input to 4 feature maps of size 26×26 using four trainable 3×3 filters.
- conv2: Depthwise 2D convolution with 4 groups. Processes each of the 4 maps using its own 3×3 filter, producing 4 feature maps of size 24×24 (576 elements per map).
- Asymmetrical Factor Slicing: Rather than using complex pooling layers, the 576-element grid is reshaped into 9 parallel aspect ratio configurations: (1×576) , (2×288) , (4×144) , (8×72) , (24×24) , (48×12) , (96×6) , (192×3) , and (576×1) . Each of the 36 sliced channels (9 configurations $\times$ 4 channels) is projected to 12 features using a trainable matrix $\mathbf{W}_{\text{trainable}}$ (shape $36 \times 576 \times 12$), followed by a ReLU activation, yielding 432 features.

2. Stream 2: Fixed Edge-Detection Stream: Convolves the input image with 4 static 3×3 Sobel edge-detection kernels (Sobel X, Sobel Y, Diagonal 1, and Diagonal 2), producing 4 feature maps of size 26×26 . Row-wise summation collapses each map to a 26-element vector, which is projected to 12 features using a fixed projection matrix $\mathbf{W}_{\text{fixed}}$ (shape $4 \times 26 \times 12$), followed by a ReLU activation, yielding 48 features.

The outputs are concatenated into a 480-dimensional vector ($432 + 48 = 480$), which is forwarded to the classifier head. The classifier consists of a hidden dense layer **H2** (480×128) with a ReLU, and a final linear layer **H3** (128×47) that computes EMNIST ByMerge classification scores.

6.2 Memory Optimization: Const Flash Storage

The SAM3X8E microcontroller features 512 KB of Flash memory but only 96 KB of SRAM. Allocating the model’s 317,608 INT8 parameters in SRAM would cause immediate stack overflow. To address this, all weights and scale constants are declared with the ‘const’ qualifier. This forces the compiler to store the arrays in Flash memory.

During inference, weights are read directly from Flash, and intermediate activation buffers are allocated as static global arrays in SRAM. The total static SRAM consumption is limited to ≈ 35 KB, leaving ample memory for stack allocation and system variables.

7 Conclusion

This project successfully demonstrates the design and integration of a real-time, closed-loop smart writing system. By combining hardware-level register operations, robust Reed-Solomon Forward Error Correction over $GF(2^8)$, and a Mahony attitude filter with linear velocity detrending, we achieve drift-free trajectory reconstruction and low-latency transmission. Furthermore, the implementation of post-training symmetric INT8 quantization, coupled with EMNIST-aligned knowledge distillation, enables the 317,608-parameter DAFNet student model to run real-time inference on an Arduino Due in just 0.64 ms, preserving a high classification accuracy of 89.36%. This demonstrates that rigorous algorithmic optimizations can bridge the gap between complex deep learning models and resource-constrained microcontrollers.